

### Explain the use of LAG and LEAD analytic functions and provide a real-world scenario.

These functions let you look backwards or forwards a specific number of rows in your result set without complex self-joins. LAG retrieves a value from a row that precedes the current row. LEAD retrieves a value from a row that follows the current row. They are crucial for time-series analysis like comparing today's sales to yesterday's sales.

Scenario (Banking): Comparing the current month's account balance (LAG) to the previous month's balance to calculate the change.  
 SELECT month, balance, balance - LAG(balance, 1, 0) OVER (ORDER BY month) AS balance\_change FROM account\_history;

### How do you calculate a Moving Average of the last 7 days of sales using window functions?

This asks how to define a rolling window of rows to calculate an aggregate (like average) as you move through the dataset. Use the AVG() analytic function with the ROWS BETWEEN clause to define the window size. ROWS BETWEEN 6 PRECEDING AND CURRENT ROW defines a 7-day window.

SELECT sale\_date, total\_sales, AVG(total\_sales) OVER (ORDER BY sale\_date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS moving\_avg\_7\_day FROM daily\_sales;

### When must you use a Recursive CTE (WITH clause), and how do you structure it?

This is for complex scenarios where a query must process data iteratively, like traversing an unknown depth. A Recursive CTE is used for traversing data structures with unlimited depth (e.g., organizational charts, bills of materials). It requires two parts: the Anchor Member (initial query) and the Recursive Member (which joins back to the CTE result).

Scenario: Finding all subordinates for a given Manager ID in an organization. WITH emp\_hierarchy (id, name, mgr\_id, lvl) AS (SELECT emp\_id, name, mgr\_id, 1 FROM employees WHERE emp\_id = 100 UNION ALL SELECT e.emp\_id, e.name, e.mgr\_id, h.lvl + 1 FROM employees e JOIN emp\_hierarchy h ON e.mgr\_id = h.id) SELECT name, lvl FROM emp\_hierarchy;

### Explain the MERGE statement and its primary advantage over separate DML.

Asks how to perform a single-pass INSERT or UPDATE conditionally. MERGE (or Upsert) atomically combines conditional INSERT and UPDATE operations into one statement based on a match condition. Advantage: It reduces network round trips, eliminates the risk of race conditions, and ensures better performance in batch processing.

Scenario: Synchronizing a daily transaction summary table with new and existing data. MERGE INTO target\_txn T USING source\_txn S ON (T.txn\_id = S.txn\_id) WHEN MATCHED THEN UPDATE SET T.amount = S.amount WHEN NOT MATCHED THEN INSERT (txn\_id, amount) VALUES (S.txn\_id, S.amount);

### Describe the CONNECT BY NOCYCLE clause and why it is important in hierarchical queries.

Asks how to prevent an infinite loop when processing data that might contain a recursive relationship (a cycle). CONNECT BY NOCYCLE prevents the hierarchical query from entering an infinite loop when a loop or cycle exists in the data (e.g., Employee A reports to B, and B reports to A). It stops traversing when it detects a node it has already visited in the current path. Scenario: Finding the reporting structure where an employee was accidentally set as their own great-grandparent's manager.

### What is the difference between a Weak Ref Cursor and a Strong Ref Cursor?

This checks your knowledge of cursor typing in PL/SQL. A Strong Ref Cursor is defined with a specific, rigid RETURN type (e.g., RETURN emp%ROWTYPE), enforcing type safety at compile time. A Weak Ref Cursor (using SYS\_REFCURSOR) is defined without a specific return type, offering flexibility (can open any query), but sacrificing compile-time type safety. Scenario: Use a Weak Ref Cursor in a generic reporting package that returns data from different tables based on input parameters.

### How would you tune a query that performs a Full Table Scan (FTS) on a large table despite having an index?

Asks why indexes might be ignored and what steps to force their use or improve performance otherwise. An FTS is often chosen if the query filter is unselective (returns > 10-15% of rows), the column is manipulated by a function (WHERE TRUNC(date\_col) ...), or statistics are stale. Tuning steps: 1. Collect fresh stats. 2. Use a Function-Based Index if a function is necessary. 3. Use an Index Hint (as a last resort).  
 SELECT /\*+ INDEX(e emp\_idx\_dept) \*/ \* FROM employees e WHERE dept\_id = 10;

### When is a Function-Based Index (FBI) necessary, and what is its main limitation?

Checks understanding of indexing on calculated values. An FBI computes the value of a function (e.g., UPPER(last\_name)) and stores the result in the index. It's needed when a column is always queried using a function that prevents a standard index scan. Limitation: The query must use the exact same function on the column for the FBI to be utilized. Scenario: Indexing a name column to support case-insensitive searches (e.g., WHERE UPPER(name) = 'SMITH').

### Explain the use of a Materialized View (MV) and the difference between ON COMMIT and ON DEMAND refresh.

Asks about pre-calculating results for complex queries to improve reporting speed.

An MV is a schema object that contains the results of a query, physically stored and maintained as a separate table. It boosts performance for complex aggregations. ON COMMIT: Refreshes the MV automatically when a transaction commits changes to the base tables. ON DEMAND: Refreshes only when explicitly requested (e.g., nightly job).  
Scenario: A daily sales report aggregating data across 10 tables. Refresh it ON DEMAND nightly instead of calculating the aggregation for every user query.

10

### **Describe the two main types of Joins typically seen in an execution plan (Hash vs. Nested Loops).**

Asks for knowledge about how the optimizer physically combines data sets.

Nested Loops Join: Efficient when one data set (driving table) is small and the join column is indexed. It works like a loop: for each row in the outer table, it scans the inner table's index. Hash Join: Efficient for large, non-indexed tables. It builds a hash table from the smaller data set and probes it with the larger data set.

Interpretation: If joining large tables without good indexes, a Hash Join is usually preferred. If joining a small table to a large indexed table, Nested Loops is often faster.

11

### **How can a Correlated Subquery negatively impact query performance, and what is a common way to rewrite it?**

Asks about a subquery that runs repeatedly and the technique to optimize it.

A Correlated Subquery is executed once for every single row processed by the outer query, leading to significant performance degradation on large tables. The common optimization is to rewrite it as a JOIN (often a LEFT JOIN or an INNER JOIN), allowing the optimizer to find a more efficient single access path.

Bad Query: `SELECT name FROM employees e WHERE salary > (SELECT AVG(salary) FROM employees WHERE dept_id = e.dept_id);` (Subquery runs for every employee).

12

### **Describe the difference between ROW\_NUMBER() and RANK() in an HR scenario.**

Checks precise understanding of how these ranking functions handle ties.

ROW\_NUMBER() assigns a unique sequential integer to each row within its partition, regardless of ties. RANK() assigns the same rank to rows with the same values (ties), and then skips the next sequential rank (e.g., 1, 1, 3).

Scenario (HR): Finding the single top-paid employee in each department. ROW\_NUMBER() = 1 is used to break ties arbitrarily and get exactly one record.

13

### **Describe the Package Specification versus the Package Body and the concept of Overloading.**

Checks understanding of PL/SQL modularity and flexible function signatures.

The Specification is the interface, declaring public procedures, functions, and variables visible to the outside world. The Body contains the implementation details and private logic. Overloading allows you to create multiple procedures/functions with the same name within a package, as long as they have different parameter counts or data types.

Scenario: A package called ACCOUNT\_UTIL with two CALCULATE\_INTEREST functions: one accepting (account\_id, rate), and another accepting (account\_id).

14

### **How does BULK COLLECT improve performance, and what is the typical limiting clause used with it?**

Asks how to reduce network communication between the database and PL/SQL engines.

BULK COLLECT fetches multiple rows into a PL/SQL collection (array) with a single round trip, greatly reducing context switching overhead. The typical limiting clause is LIMIT (or FETCH FIRST n ROWS in the cursor loop) to fetch records in batches, controlling memory usage.

`FETCH c1 BULK COLLECT INTO l_emp_names LIMIT 1000;`

15

### **Explain the advantage of using FORALL with the SAVE EXCEPTIONS clause.**

Checks advanced error handling during bulk DML operations.

FORALL executes a single DML statement (INSERT/UPDATE/DELETE) on a collection of rows in a batch. SAVE EXCEPTIONS ensures that if some records fail (e.g., duplicate key), the successful ones are still processed. The failed records and their error codes can then be retrieved from SQL%BULK\_EXCEPTIONS.

Scenario: Inserting 10,000 transaction records where 50 might fail due to a primary key violation. You want to process the 9,950 successful ones.

16

### **Differentiate between an Associative Array and a Nested Table in PL/SQL programming.**

Asks about collection memory and indexing.

Associative Array (Index-by Table): Used only within PL/SQL; indexed by an integer or string (non-sequential indexing is common); stored in the server's PGA memory. Nested Table: Can be stored in the database; indexed by sequential integers starting at 1; usually stored out-of-line (separate storage segment).

Use Case: Associative Array is best for lookups using non-integer keys (e.g., using a product code as the index). Nested Table is best when the collection needs to be persistent or passed to the database as a column.

17

### **When must you use EXECUTE IMMEDIATE for Dynamic SQL, and how do you mitigate the risk of SQL Injection?**

Checks ability to run code whose structure isn't known until runtime.

Dynamic SQL is necessary when the structure of the SQL (table name, column list, WHERE clause structure) is variable or created at runtime. Mitigation: Always use Bind Variables (USING clause) for all data inputs to separate the data from the SQL command text, preventing malicious code execution.

```
`EXECUTE IMMEDIATE 'UPDATE '
```

18

### **What is an Autonomous Transaction, and provide a use case.**

Asks about independent transaction control.

An Autonomous Transaction (declared with PRAGMA AUTONOMOUS\_TRANSACTION) runs in its own session, independent of the main transaction. Its actions (COMMIT/ROLLBACK) do not affect the calling transaction. Use Case: Error Logging. If a main procedure fails and rolls back, the error log entry (which should be in an autonomous transaction) is still committed to the log table.

```
CREATE PROCEDURE log_activity (p_msg IN VARCHAR2) IS PRAGMA AUTONOMOUS_TRANSACTION; BEGIN INSERT INTO audit_log (message) VALUES (p_msg); COMMIT; END;
```

19

### **Explain the purpose of a Compound Trigger and how it improves code efficiency.**

Checks knowledge of a trigger type that combines multiple events.

A Compound Trigger combines the logic for multiple timing points (BEFORE STATEMENT, BEFORE ROW, AFTER ROW, AFTER STATEMENT) into a single, unified PL/SQL block. Efficiency: It allows sharing global variables across the different timing points, which is essential for tasks like accumulating changed data in a collection during BEFORE ROW and processing the entire collection during AFTER STATEMENT in one batch.

Use Case: Complex auditing where you need to track changes (:NEW, :OLD) in a collection during row processing and then write one single consolidated audit record after the statement finishes.

20

### **Differentiate between WHEN-NEW-FORM-INSTANCE and PRE-FORM triggers in Oracle Forms.**

Checks understanding of the Form's runtime lifecycle.

PRE-FORM: Fires before the form is loaded into memory. It is rarely used, mainly for pre-processing tasks before the application environment is set up. WHEN-NEW-FORM-INSTANCE (WNFI): Fires after the form is loaded, data blocks are instantiated, and the first item is navigated to. This is the primary trigger for setting initial values, enabling/disabling items, and running startup logic.

Use Case (Banking): Use WNFI to query the user's role and conditionally hide the "Approve Transaction" button upon form startup.

21

### **In Forms, explain the role of a Record Group when defining a List of Values (LOV).**

Asks about the source of data for the selection window.

The Record Group is the source query that defines the data displayed in the LOV selection window. It is the structure (column names and data types) that the LOV definition points to. The LOV then maps the columns in the Record Group to the columns in the LOV display grid.

Use Case: Defining a Record Group named RG\_CUSTOMERS with CUSTOMER\_ID, NAME, ADDRESS to populate a customer selection LOV.

22

### **Describe the flow of logic and typical use case for a WHEN-VALIDATE-ITEM (WVI) trigger.**

Asks about the primary trigger for enforcing field-level business rules.

WVI fires when the user leaves a data item, before the value is committed to the data block record. Flow: 1. User exits item. 2. WVI fires. 3. If validation fails, use RAISE FORM\_TRIGGER\_FAILURE. 4. If successful, the logic completes. Use Case: Ensuring a mandatory field (like Account\_Number) is populated, or checking that the End\_Date is greater than the Start\_Date.

```
IF :BLOCK.END_DATE < :BLOCK.START_DATE THEN MESSAGE('End date must be after start date.');
```

```
RAISE FORM_TRIGGER_FAILURE; END IF;
```

23

### **Explain the use of the POST-QUERY trigger and the risk associated with its execution.**

Checks understanding of modifying fetched data and performance considerations.

POST-QUERY fires once for every record fetched into a data block after the main query has executed. Use Case: To fetch a description (e.g., DEPT\_NAME) from a lookup table based on an ID (DEPT\_ID) that was fetched in the main query, and display it in a non-database item. Risk: Because it fires for every row, inefficient SQL inside this trigger can severely degrade performance when querying large record sets.

```
SELECT dept_name INTO :BLOCK.DEPT_NAME_DISPLAY FROM departments WHERE dept_id = :BLOCK.DEPT_ID;
```

24

### **What is the purpose of the KEY-COMMIT trigger, and when would you need to modify the default commit behavior?**

Asks how to customize the transaction submission process.

KEY-COMMIT fires when the user executes the "Commit" action (e.g., clicking a Save button). The default behavior is to commit pending changes. You modify it to: 1. Add custom validation before saving. 2. Execute custom PL/SQL (e.g., auditing). 3. Implement multi-form commits (rare). The trigger logic usually ends with an explicit COMMIT\_FORM;

Use Case (Transaction Module): Before committing a withdrawal, you would use a KEY-COMMIT to check the current account balance via a stored function to ensure sufficient funds.

25

### **Explain the difference between PRE-INSERT and ON-INSERT data block triggers.**

Checks understanding of customizing the default DML processing.

PRE-INSERT: Fires before Oracle Forms attempts the default INSERT into the base table. Used to set default values (e.g., Created\_Date) or perform last-minute validations. ON-INSERT: Overrides the default DML processing. You must provide the entire custom INSERT statement (usually using EXECUTE IMMEDIATE or a stored procedure) yourself.  
Use Case: Use ON-INSERT when the data needs to be inserted into a remote database via a database link, bypassing the default local table insert.

26

### **How do you use a PL/SQL Library (PLL) in Oracle Forms, and what benefit does it provide?**

Asks about code reusability across multiple form modules.

A PLL is a file containing common, reusable PL/SQL code (functions, procedures) that can be attached to multiple forms. Benefit: It promotes code consistency and reusability. A change made once in the PLL is immediately reflected across all attached forms when the library is compiled and the forms are re-run.

Use Case: A F\_VALIDATE\_TAX\_ID function used across 20 different customer and vendor forms should be placed in a single PLL.

27

### **Describe a situation where you would need a System Trigger (like AFTER LOGON) at the database level.**

Asks about advanced database-wide event handling.

System Triggers fire on DDL or database events (logon, logoff, server startup). Use Case: To enforce security or session settings. For example, an AFTER LOGON ON DATABASE trigger can be used to set the CLIENT\_IDENTIFIER or to set resource limits for certain users upon connection to the database.

```
CREATE TRIGGER set_app_context AFTER LOGON ON DATABASE BEGIN
DBMS_APPLICATION_INFO.SET_CLIENT_INFO(SYS_CONTEXT('USERENV', 'OS_USER')); END;
```

28

### **Explain the difference between a Page Rendering Process and a Page Submission Process in APEX.**

Asks about the timing and purpose of APEX PL/SQL execution.

Rendering Process: Executes on the server before the page is shown to the user (e.g., setting default item values, complex authorizations). Submission Process: Executes on the server after the user clicks a button and submits the page (e.g., DML logic like INSERT/UPDATE, calling stored procedures).

Scenario: Use a Rendering Process to fetch and display the current inventory count. Use a Submission Process to save the updated inventory count.

29

### **How do you pass data from a PL/SQL Dynamic Action back to a client-side JavaScript function for display?**

Checks knowledge of the bi-directional communication in APEX.

The PL/SQL code in the Dynamic Action should populate the result into a Page Item (e.g., :P1\_RESULT). Then, the follow-on client-side JavaScript action uses the built-in APEX function \$v('P1\_RESULT') to access the item's new value.

Scenario: A user enters a customer ID. A PL/SQL Dynamic Action fetches the customer's loyalty status and passes it back to JavaScript to display a popup alert.

30

### **Describe the function of an APEX Collection, and provide a use case where you would use it instead of a temporary table.**

Checks knowledge of a key APEX session-state data structure.

An APEX Collection is a temporary, session-specific data structure that stores rows of data (up to 50 attribute columns) identified by a name. Use Case: Shopping Cart or Multi-step Wizards. It's better than a temporary table because it is inherently tied to the user session, avoids table locking, and requires no DDL/cleanup logic.

```
APEX_COLLECTION.CREATE_COLLECTION_FROM_QUERY (p_collection_name => 'CART', p_query => 'SELECT item_id,
qty, price FROM items WHERE user_id = :APP_USER');
```

31

### **How is Session State managed in APEX, and why is it crucial to use Bind Variables (:P1\_ITEM) in SQL/PLSQL instead of substitution strings (&P1\_ITEM.)?**

Asks about how APEX stores and handles user data.

APEX stores Session State (the values of all page and application items) in the database in the WWV\_FLOW\_DATA or similar table. Crucial: Using Bind Variables (:P1\_ITEM) ensures the SQL engine can reuse the execution plan (since the query text is constant), preventing SQL Injection and improving performance. Substitution strings (with &) should only be used where the object name (like a column or table) is dynamic.

Scenario: Use :P1\_ORDER\_ID in the WHERE clause of a report's SQL query.

32

### **Explain the Oracle Forms Runtime Process from user login to data display.**

Checks understanding of the client-server interaction architecture.

1. User Request: User connects via browser. 2. Forms Server: Forms Listener receives the request and starts a dedicated Java client session. 3. Forms Runtime: The server loads the .fmx module and executes the PRE-FORM and WHEN-NEW-FORM-INSTANCE triggers. 4. Initial Data: Data blocks execute their default query or logic, and the form is rendered on the client side.  
Key Components: Forms Server (WebLogic or Standalone), Forms Listener, and Forms Runtime Process.

33

### **How do you manage a complex cascading LOV structure in Forms (e.g., Country → State → City)?**

Checks ability to dynamically filter an LOV based on a parent field's value.

Use the WHEN-VALIDATE-ITEM (WVI) trigger on the parent item (Country). Inside the WVI, use the SET\_ITEM\_PROPERTY built-in to modify the Record Group query of the child LOV (State) to include a WHERE clause filtered by the parent's current value (:BLOCK.COUNTRY). Then, call POPULATE\_LIST if it's a list item, or rely on the LOV to execute the new query on open.  
`SET\_LOV\_PROPERTY('LOV\_STATE', GROUP\_NAME, 'RG\_STATE'); SET\_BLOCK\_PROPERTY('RG\_STATE',  
QUERY\_TEXT, 'SELECT state\_id, state\_name FROM states WHERE country\_id = '

34

### **What is the difference between a Formula Column and a Summary Column in Oracle Reports?**

Asks about calculated fields in the report output.

Formula Column: Calculates a value based on other columns in the same record (e.g., Price \* Quantity). Summary Column: Calculates an aggregate value (e.g., SUM, AVG, COUNT) across multiple records, usually based on a Group break (e.g., total sales per Region).

Report Example: A report showing customer transactions. The Formula Column calculates the Tax on each transaction. The Summary Column calculates the total transactions for the entire report.

35

### **Explain the function of a Lexical Parameter in Oracle Reports, and why it is dangerous to use.**

Checks understanding of dynamic query modification.

A Lexical Parameter substitutes a string value directly into the SQL query text at runtime, allowing dynamic modification of clauses like SELECT, FROM, or ORDER BY. Danger: It is highly susceptible to SQL Injection, as the user-provided string is concatenated directly into the query without binding. Use Case: Dynamically changing the ORDER BY clause of a report.

SELECT col1, col2 FROM my\_table &P\_WHERE\_CLAUSE ORDER BY &P\_SORT\_COLUMN;

36

### **In Forms, how do you navigate programmatically between items and blocks, and what is the key built-in used?**

Asks about controlling the user's focus.

The primary built-in function is GO\_ITEM('BLOCK.ITEM\_NAME') to set focus to a specific item. GO\_BLOCK('BLOCK\_NAME') moves focus to a block without setting item focus (usually the first navigable item). The KEY-NEXT-ITEM trigger is often customized to alter the default navigation path.

Use Case (Account Opening): After validating and committing data in the CUSTOMER\_DETAIL block, you use GO\_BLOCK('ACCOUNT\_DETAIL'); to send the user to the next step.

37

### **What is a Canvas in Forms, and what is the difference between a Content and a Stacked canvas?**

Checks understanding of the visual structure of a form.

A Canvas is the surface upon which data blocks and items are placed for display. Content Canvas: The main, base window of the form. Only one content canvas is fully visible at a time. Stacked Canvas: A smaller canvas that overlays the content canvas, used for popups, tabs, or modal windows.

Visual: The main account entry screen is on a Content Canvas. A window that pops up to show historical notes for the account is on a Stacked Canvas.

38

### **How do you ensure the user is prompted to save changes if they try to close the form without committing?**

Checks knowledge of default form behavior and key system variables.

Oracle Forms handles this automatically by setting the SYSTEM.FORM\_STATUS variable to CHANGED whenever a data block item is modified. If a user attempts to exit (e.g., via KEY-EXIT or closing the window) while SYSTEM.FORM\_STATUS is CHANGED, Forms prompts the user with "Do you want to save the changes you have made?".

You can customize this behavior in the KEY-EXIT trigger, but typically you rely on the default behavior.

39

### **Describe a Forms trigger where you would set the value of a Non-Database Item, and what function you would use to fetch the value.**

Asks about querying data into fields that aren't tied to the base table.

The best trigger is the POST-QUERY trigger. Function: Use the standard PL/SQL SELECT INTO statement, which is valid within Forms PL/SQL units.

Use Case (Transaction Module): After fetching a transaction record (T\_ID, CUST\_ID), use POST-QUERY to SELECT cust\_name INTO :BLOCK.CUSTOMER\_NAME\_DSP FROM customers WHERE cust\_id = :BLOCK.CUST\_ID;

40

### **How do you ensure that a sensitive field, like a password or a national ID, is never displayed on the screen after the initial entry in Forms?**

Asks about item security properties.

Use the SET\_ITEM\_PROPERTY built-in function to change the field's VISIBLE or DISPLAYED properties. Alternatively, you can clear the item's value in a POST-CHANGE trigger or use a Password Item which masks the value.

Scenario: After the user enters the PIN, use SET\_ITEM\_PROPERTY('BLOCK.PIN\_FIELD', VISIBLE, PROPERTY\_FALSE); or simply clear the value: :BLOCK.PIN\_FIELD := NULL; after storing it.